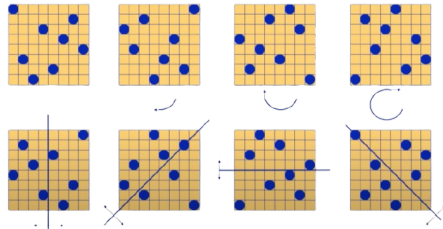


L05: Symmetry and dominance



Prof. Tias Guns and Dr. Hendrik 'Henk' Bierlee

KU LEUVEN

Partly based on slides from Pierre Flener and Dimos Tsouros.

Outline

1. Introduction

2. Symmetries

3. Symmetry Breaking

Symmetry Breaking by Reformulation

Symmetry Breaking by Constraints

4. Dominance Breaking

5. Recap

Quick Recap

1. **Modelling**: express problem in terms of
 - ▶ parameters,
 - ▶ decision variables, with their domains
 - ▶ constraints, and
 - ▶ (optionally) objective.
2. **Solving**: solve using a state-of-the-art solver.

From a Problem to a Model

What is a good model for a constraint problem?

- ▶ A model that **correctly** represents the problem.
- ▶ A model that is **easy** to understand and maintain.
- ▶ A model that is solved **efficiently**.

Modelling

Modelling is still more a craft than a science:

- ▶ Choice of **viewpoint**: which are the decision variables and their domains?
 - ▶ Use of **auxiliary** variables ...
 - ▶ **Channelling** different viewpoints ...
- ▶ Choice of the constraint formulations
 - ▶ Use **redundant** constraints to reduce the search space ...
- ▶ Choice of solver and solver configuration

Make a model correct before making it efficient!

Making an efficient model

Redundant constraints can reduce the search space.

Definition

Redundant constraints, also called *implied* constraints, are constraints which are entailed by the constraints defining the problem. They do not change the set of solutions, and hence are logically redundant.

Is there any other way to **reduce the search space**, making the model more efficient?

Outline

1. Introduction

2. Symmetries

3. Symmetry Breaking

Symmetry Breaking by Reformulation

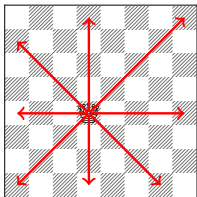
Symmetry Breaking by Constraints

4. Dominance Breaking

5. Recap

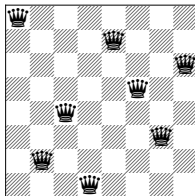
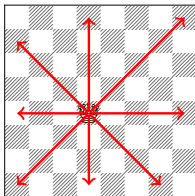
The NQUEENS problem

To place n queens on an $n * n$ chessboard such that they do not attack each other (i.e. share a row, column, or diagonal)



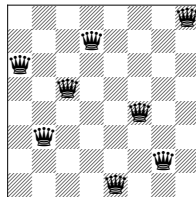
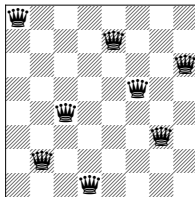
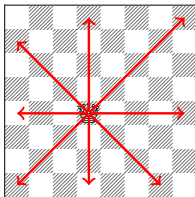
The NQUEENS problem

To place n queens on an $n * n$ chessboard such that they do not attack each other (i.e. share a row, column, or diagonal)



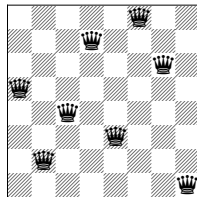
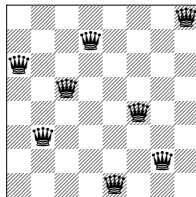
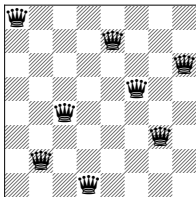
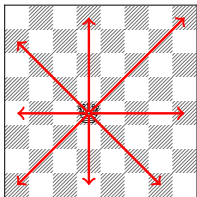
The NQUEENS problem

To place n queens on an $n * n$ chessboard such that they do not attack each other (i.e. share a row, column, or diagonal)



The NQUEENS problem

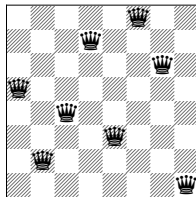
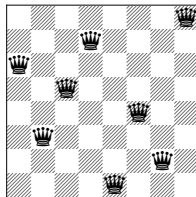
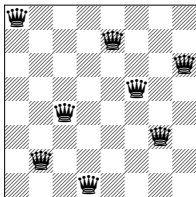
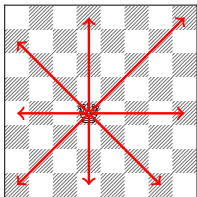
To place n queens on an $n * n$ chessboard such that they do not attack each other (i.e. share a row, column, or diagonal)



How many solutions are there?

The NQUEENS problem

To place n queens on an $n * n$ chessboard such that they do not attack each other (i.e. share a row, column, or diagonal)

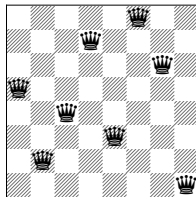
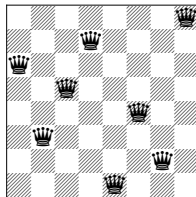
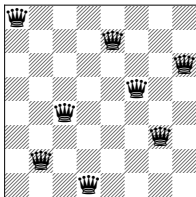
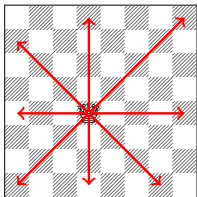


How many solutions are there? 92.

How many non-symmetrical (w.r.t rotation/reflection) solutions are there?

The NQUEENS problem

To place n queens on an $n * n$ chessboard such that they do not attack each other (i.e. share a row, column, or diagonal)



How many solutions are there? 92.

How many non-symmetrical (w.r.t rotation/reflection) solutions are there? 12.

Symmetries

Symmetries are defined based on the concept of **permutation**; permuting elements of a group of variables or values can lead to symmetric solutions!

Definition

A **permutation** π over a discrete set D is a 1-to-1 function from D to D . It is an arrangement of the elements of the set in a specific order.

■ Intuitively: just a re-ordering of the elements, e.g.:

$$\begin{array}{l} i : (1 \quad 2 \quad 3 \quad 4 \quad 5) \\ \pi(i) : (4 \quad 3 \quad 2 \quad 1 \quad 5) \end{array}$$

Symmetries

Definition

A **symmetry** σ is a permutation of values or decision variables (or both) that **preserves all constraints and solutions**: it transforms (partial) solutions into (partial) solutions, and it transforms (partial) non-solutions into (partial) non-solutions.

Example

Assume a problem with 5 variables, that has 1 variable symmetry σ :

$$\begin{aligned} X &: (x_1 \ x_2 \ x_3 \ x_4 \ x_5) \\ \sigma(X) &: (x_4 \ x_3 \ x_2 \ x_1 \ x_5) \end{aligned}$$

Based on the symmetry σ , each assignment v can be transformed to a symmetric assignment:

$$\begin{aligned} x = v &: (x_1 = v_1 \ x_2 = v_2 \ x_3 = v_3 \ x_4 = v_4 \ x_5 = v_5) \\ \sigma(x) = v &: (x_4 = v_1 \ x_3 = v_2 \ x_2 = v_3 \ x_1 = v_4 \ x_5 = v_5) \end{aligned}$$

If $x = v$ is a solution, then so is $\sigma(x) = v$. If $x = v$ is **not** a solution, then $\sigma(x) = v$ is **not** a solution.

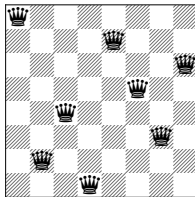
A model for the NQUEENS problem

A choice of model uses the fact that all n queens need separate rows: we can use a single integer variable $q_i \in [1..n]$ per queen to denote that the queen in row i is at column q_i .

$$\begin{aligned} & q_i \in [1..n], \forall i \in [1..n] \\ & \text{ALLDIFFERENT}(\{q_i : i \in [1..n]\}), \\ & \text{ALLDIFFERENT}(\{q_i - i : i \in [1..n]\}), \\ & \text{ALLDIFFERENT}(\{q_i + i : i \in [1..n]\}) \end{aligned}$$

A solution:

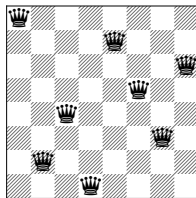
$$[q_1 = 1, q_2 = 5, q_3 = 8, q_4 = 6, q_5 = 3, q_6 = 7, q_7 = 2, q_8 = 4]$$



A model for the NQUEENS problem

A choice of model uses the fact that all n queens need separate rows: we can use a single integer variable $q_i \in [1..n]$ per queen to denote that the queen in row i is at column q_i .

```
1 n = 8 # dimension of square board
2
3 # 'q[i] == j' denotes that the queen in row 'i' is in col 'j'
4 q = cp.intvar(0, n - 1, shape=n, name="q")
5
6 # Constraints on columns, and on both diagonals
7 model = cp.Model(
8     cp.AllDifferent(q),
9     cp.AllDifferent([q[i] - i for i in range(n)]),
10    cp.AllDifferent([q[i] + i for i in range(n)]),
11 )
12 model.solve()
13 print(q.value()) # [0 4 7 5 2 6 1 3]
```



Example (Variable symmetry: NQUEENS for $n = 9$)

The symmetries of NQUEENS for $n = 9$ are:

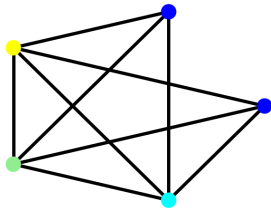
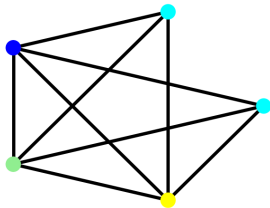
$$\text{id: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \end{pmatrix} \quad \text{r90: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_7 & q_4 & q_1 & q_8 & q_5 & q_2 & q_9 & q_6 & q_3 \end{pmatrix}$$

$$\text{r180: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_9 & q_8 & q_7 & q_6 & q_5 & q_4 & q_3 & q_2 & q_1 \end{pmatrix} \quad \text{r270: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_3 & q_6 & q_9 & q_2 & q_5 & q_8 & q_1 & q_4 & q_7 \end{pmatrix}$$

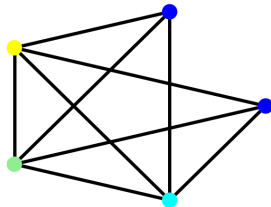
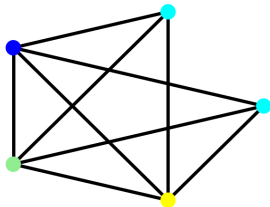
$$\text{x: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_3 & q_2 & q_1 & q_6 & q_5 & q_4 & q_9 & q_8 & q_7 \end{pmatrix} \quad \text{y: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_7 & q_8 & q_9 & q_4 & q_5 & q_6 & q_1 & q_2 & q_3 \end{pmatrix}$$

$$\text{d1: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_1 & q_4 & q_7 & q_2 & q_5 & q_8 & q_3 & q_6 & q_9 \end{pmatrix} \quad \text{d2: } \begin{pmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 & q_7 & q_8 & q_9 \\ q_9 & q_6 & q_3 & q_8 & q_5 & q_2 & q_7 & q_4 & q_1 \end{pmatrix}$$

In the **graph colouring** problem, the goal is to assign at most k different colours to nodes such that adjacent nodes do not have the same colour.



In the **graph colouring** problem, the goal is to assign at most k different colours to nodes such that adjacent nodes do not have the same colour.



Example (Value symmetry: graph colouring)

v	v_1	v_2	v_3	v_4
$\sigma(v)$	blue	teal	green	yellow
	yellow	blue	green	teal

The model where the nodes (as decision variables) take colours (as values) has $k!$ **value symmetries** because any permutation of the colours transforms a (non-)solution into another (non-)solution.

Value and variable symmetry

Definition

A **variable symmetry** σ is a permutation of the decision variables in a CSP that **preserves all constraints and solutions**. For a full assignment $(x_1 \mapsto v_1, \dots, x_n \mapsto v_n)$, it is symmetrical to $(x_{\sigma(1)} \mapsto v_1, \dots, x_{\sigma(n)} \mapsto v_n)$ (equivalently: the same values, reassigned to the permuted variables). If two variables can be swapped without changing the set of solutions or violating any constraints, the CSP exhibits variable symmetry.

Definition

A **value symmetry** σ is a permutation of the values in a CSP that **preserves all constraints and solutions**. For a full assignment $(x_1 \mapsto v_1, \dots, x_n \mapsto v_n)$, it is symmetrical to $(x_1 \mapsto \sigma(v_1), \dots, x_n \mapsto \sigma(v_n))$. If two values can be swapped without altering the set of solutions or violating any constraints, the CSP exhibits value symmetry.

The SUBSETSUM problem

In a SUBSETSUM, the goal is to find a subset of n elements from a set S , subject to some constraints. We can model this problem with n integer decision variables taking distinct values from S :

$$x_i \in S, \forall i \in [1..n]$$
$$\text{ALLDIFFERENT}(x)$$

	x_1	x_2	x_3
V_1	1	2	3
V_2	1	2	4
V_3	3	2	1
V_4	4	5	2

Figure: Four solutions for $n = 3, S = [0..5]$

The SUBSETSUM problem

In a SUBSETSUM, the goal is to find a subset of n elements from a set S , subject to some constraints. We can model this problem with n integer decision variables taking distinct values from S :

$$x_i \in S, \forall i \in [1..n]$$
$$\text{ALLDIFFERENT}(x)$$

	x_1	x_2	x_3
V_1	1	2	3
V_2	1	2	4
V_3	3	2	1
V_4	4	5	2

Figure: Four solutions for $n = 3, S = [0..5]$

But V_1 and V_3 are symmetric since they represent the same unordered set. This set representation using an ordered list has $n!$ symmetries.

The SUBSETSUM problem

In a SUBSETSUM, the goal is to find a subset of n elements from a set S , subject to some constraints. We can model this problem with n integer decision variables taking distinct values from S :

$$x_i \in S, \forall i \in [1..n]$$
$$\text{ALLDIFFERENT}(x)$$

	x_1	x_2	x_3
V_1	1	2	3
V_2	1	2	4
V_3	3	2	1
V_4	4	5	2

Figure: Four solutions for $n = 3, S = [0..5]$

But V_1 and V_3 are symmetric since they represent the same unordered set. This set representation using an ordered list has $n!$ **variable** symmetries.

Example: LATINSQUARES

Construct an $n * n$ Latin square where each row and column contains each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Example: LATIN SQUARES

Construct an $n * n$ Latin square where each row and column contains each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Swap rows 1 and 2

2	3	4	1
1	2	3	4
3	4	1	2
4	1	2	3

Example: LATIN SQUARES

Construct an $n * n$ Latin square where each row and column contains each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Swap rows 1 and 2

2	3	4	1
1	2	3	4
3	4	1	2
4	1	2	3

Swap columns 1 and 2

2	1	3	4
3	2	4	1
4	3	1	2
1	4	2	3

The problem has $n!^2$ **index symmetries** because any permutation of the rows or columns ($n!$ permutations each) transforms a (non-)solution into another (non-)solution: the indices are not distinguished.

Variable symmetry special case: Index symmetry

Index symmetry is a special case of variable symmetry, occurring on slices of a tensor of decision variables.

Definition

Index symmetry: occurs in problems involving arrays or matrices of decision variables; permutations of **slices** of an array of decision variables preserve all constraints and solutions.

Common cases: **row symmetry**, **column symmetry** ...

Careful: Index symmetries multiply up!

If there is full row and column symmetry in an $m \times n$ array (that is, if there are $m!$ row symmetries and $n!$ column symmetries), then there are $m! \cdot n!$ compositions of symmetries.

Symmetries

What is the origin of the symmetries?

Symmetries can be present due to different reasons

- ▶ Problem symmetries
- ▶ Instance symmetries
- ▶ Model symmetries

Problem symmetries

Definition

A **problem symmetry** is a symmetry of a CSP that is inherent to the problem itself and is detectable in *every model* of the problem.

The symmetries in graph colouring and n -queens we discussed are **problem symmetries**:

- ▶ The values in graph colouring do not represent entities with different attributes; the actual colour does not matter.
- ▶ The variables in n -queens represent the queens with the same characteristics.
- ▶ Both the tasks and the machines in the scheduling problem are indistinguishable, thus symmetric.

Instance symmetries

Definition

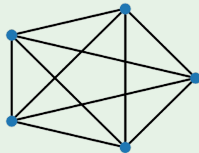
An **instance symmetry** is a symmetry detectable in a specific instance of a CSP. These symmetries depend on the particular values or structure of the instance rather than the general problem or its model.

Example (Graph colouring)

Problem: Use k colours to paint the nodes of a graph such that no adjacent nodes have the same colour.

Instance: Use 5 colours to paint 5 nodes that are all adjacent to each other (i.e. that form a clique).

Symmetry: Besides the **value symmetry** that is inherent to the graph colouring problem, we also have **variable symmetry** in this instance due to its structure: All variables have the same properties and thus are symmetric!



Model symmetries

Definition

A **model symmetry** is a symmetry that arises from the chosen viewpoint of the problem, and is not detectable in every model of the problem.

The symmetries in the subset model are *not* problem symmetries but **model symmetries**: they are *not* present for every modeling choice.

Example

For the **SUBSETSUM**, an alternative viewpoint might not have such symmetries.

Model symmetries are **introduced** during modeling and can be avoided by using a different model (to be discussed further).

Symmetry Breaking

Definition

A **symmetry class** is an equivalence class of solutions under all the considered symmetries, including their compositions.

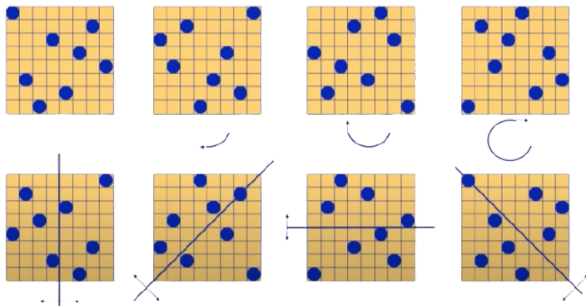


Figure: For NQUEENS, the symmetry class contains all solutions we can get by applying **rotational** symmetries (top row) and **reflection** symmetries (bottom row)

Outline

1. Introduction
2. Symmetries
- 3. Symmetry Breaking**
 - Symmetry Breaking by Reformulation
 - Symmetry Breaking by Constraints
4. Dominance Breaking
5. Recap

Why do we care about symmetries?

Observation:

- ▶ A found **solution** can be transformed without search into a symmetric solution in polynomial time (i.e. much faster than **searching** for them).
- ▶ **More importantly**: a solver may waste a lot of effort on backtracking over gazillions of (partial) **non-solutions** that are symmetric to already visited ones

Symmetries affect the solver's performance

Symmetry handling has two aspects:

- ▶ **Symmetry Detection**: Detect (some) symmetries if present
- ▶ **Symmetry Breaking**: Prevent or exploit the detected symmetries rather than needlessly finding them through solving

Symmetry Breaking

Breaking symmetries can be done in different ways.

Types of symmetry breaking:

- ▶ **Symmetry breaking by reformulation**: the elimination of the model symmetries by changing viewpoints
- ▶ **Static symmetry breaking**: eliminate symmetric solutions by adding **constraints** upfront
- ▶ **Dynamic symmetry breaking**: the elimination of symmetric solutions dynamically during the solver's **search** (out of scope)

Symmetry Breaking

Regardless of the type of symmetry breaking used, our method needs to be **sound**, **strong**, and **efficient**.

- ▶ **Soundness**: Symmetry breaking must keep ***at least one*** solution from each symmetry class.
- ▶ **Strength**: Symmetry breaking should keep as few solutions as possible from each symmetry class: **ideally only one!**
- ▶ **Efficiency**: The overhead of detecting and breaking symmetries should not outweigh the performance benefits

Outline

1. Introduction
2. Symmetries
- 3. Symmetry Breaking**
 - Symmetry Breaking by Reformulation
 - Symmetry Breaking by Constraints
4. Dominance Breaking
5. Recap

Breaking model symmetries by reformulation

Definition (recap)

A **model symmetry** is a symmetry that arises from a specific representation of a CSP and is not detectable in every model of the problem.

We can break such symmetries by reformulating the model!

Example

In the SUBSETSUM, we represented a set S by a list of integer variables $x_i \in S$, e.g. for $n = 3$ and $S = \{1, 2, 3, 4, 5\}$, the subset $\{1, 2, 4\}$ is represented by $[1, 2, 4]$, and by $[4, 2, 1]$, and by $[1, 4, 2]$, ...

Instead, an alternative representation uses $|S|$ Boolean variables, with $B_i = 1$ representing iff the i -th element of S is chosen. This avoids all symmetries, since now $\{1, 2, 4\}$ is **uniquely** represented by $[1, 1, 0, 1, 0]$.

Outline

1. Introduction

2. Symmetries

3. Symmetry Breaking

Symmetry Breaking by Reformulation

Symmetry Breaking by Constraints

4. Dominance Breaking

5. Recap

Symmetry Breaking by Constraints

Often, we cannot break all symmetries by reformulation (or perhaps the reformulation has other downsides). Instead, we can add **symmetry-breaking constraints** to eliminate any remaining symmetries.

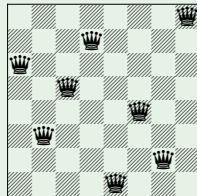
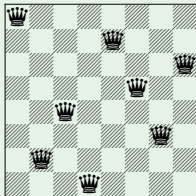
- ▶ Are symmetry-breaking constraints **redundant constraints**?

Symmetry Breaking by Constraints

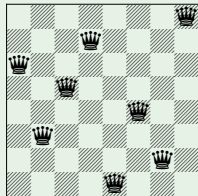
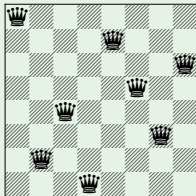
Often, we cannot break all symmetries by reformulation (or perhaps the reformulation has other downsides). Instead, we can add **symmetry-breaking constraints** to eliminate any remaining symmetries.

- ▶ Are symmetry-breaking constraints **redundant constraints**? **No, as they change the set of solutions.**
- ▶ However, breaking symmetries with constraints can allow for further redundant constraints

Example (NQUEENS)

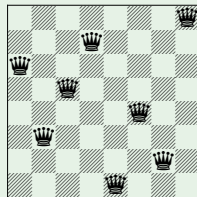
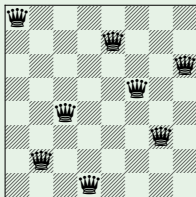


Example (NQUEENS)



We can place the queen in the first row on the left half of the board:

Example (NQUEENS)

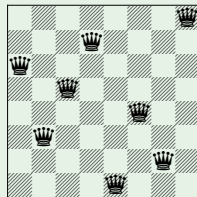
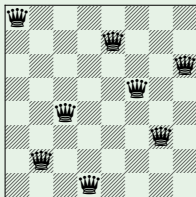


We can place the queen in the first row on the left half of the board:

```
model += q[0] <= (n // 2) - 1
```

► Is it ***sound***?

Example (NQUEENS)

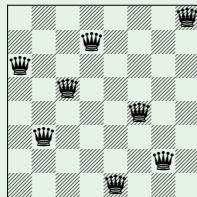
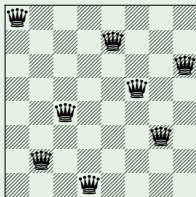


We can place the queen in the first row on the left half of the board:

```
model += q[0] <= (n // 2) - 1
```

- Is it **sound**? Yes: if the first-row queen is on the right half, vertical reflection will put it on the left.
- Is it **strong**?

Example (NQUEENS)

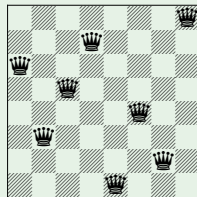
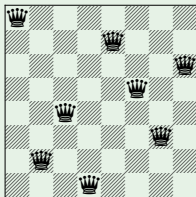


We can place the queen in the first row on the left half of the board:

```
model += q[0] <= (n // 2) - 1
```

- Is it **sound**? Yes: if the first-row queen is on the right half, vertical reflection will put it on the left.
- Is it **strong**? Breaking vertical reflection halves the 92 solutions to 46.
- Is it **efficient**?

Example (NQUEENS)



We can place the queen in the first row on the left half of the board:

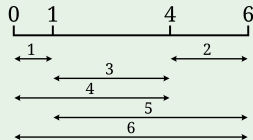
```
model += q[0] <= (n // 2) - 1
```

- Is it **sound**? Yes: if the first-row queen is on the right half, vertical reflection will put it on the left.
- Is it **strong**? Breaking vertical reflection halves the 92 solutions to 46.
- Is it **efficient**? For $n = 290$, from 29 to 21 seconds.

Ordering constraint on variables

Example (Golomb Rulers)

A Golomb ruler has m marks, where all the $\frac{m(m-1)}{2}$ distances between marks ($x_j - x_i \forall 1 \leq i < j \leq m$) are different



- ▶ Variables: one for each mark $x_1, x_2, \dots, x_m$
- ▶ Constraints: $x_j - x_i \neq x_l - x_k$ for all distinct pairs
- ▶ **Variable Symmetry**: Variables are symmetrical. Solution $[0, 1, 4, 6]$ is equivalent to $[0, 4, 1, 6]$, and $[1, 0, 4, 6]$, and ...
- ▶ **Symmetry breaking**: Enforce ordering with $x_1 < x_2 < \dots < x_m$

Or directly use the INCREASINGSTRICT(X) global constraint for symmetry-breaking!

(As in the channelled Golomb model in L04: ordering breaks variable symmetry on the marks.)

Global constraints for ordering of variables

Definition

The $\text{INCREASING}(X)$ global constraint holds if and only if n variables in X are in increasing order:

$$X_1 \leq X_2 \leq \dots \leq X_{n-1} \leq X_n$$

Other global constraints of the same family:

$\text{INCREASINGSTRICT}(X)$ $X_1 < X_2 < \dots < X_{n-1} < X_n$

$\text{DECREASING}(X)$ $X_1 \geq X_2 \geq \dots \geq X_{n-1} \geq X_n$

$\text{DECREASINGSTRICT}(X)$ $X_1 > X_2 > \dots > X_{n-1} > X_n$

Also applicable to the integer formulation of the sets in SUBSETSUM!

Symmetry-breaking by ordering constraints

A common way to break symmetries is to impose an ordering.

Breaking **variable symmetries**: ordering on the variables.

- ▶ Enforcing an order in the variables, they are no longer indistinguishable and thus not symmetric.
- ▶ May also have an effect on value symmetries: (Integer) values have an order that is now important!

Breaking **index symmetries**: ordering on the tensor slices.

- ▶ Row symmetries: ordering of rows.
- ▶ Column symmetries: ordering of columns.
- ▶ ...

Breaking index symmetries

Example (Latin Squares)

A Latin square of size $n * n$ requires that each row and column contain each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Breaking index symmetries

Example (Latin Squares)

A Latin square of size $n * n$ requires that each row and column contain each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Swap rows 1 and 2

2	3	4	1
1	2	3	4
3	4	1	2
4	1	2	3

Swap columns 1 and 2

2	1	3	4
3	2	4	1
4	3	1	2
1	4	2	3

Breaking index symmetries

Example (Latin Squares)

A Latin square of size $n * n$ requires that each row and column contain each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Swap rows 1 and 2

2	3	4	1
1	2	3	4
3	4	1	2
4	1	2	3

Swap columns 1 and 2

2	1	3	4
3	2	4	1
4	3	1	2
1	4	2	3

Lexicographic ordering of lists: $X \leq_{\text{lex}} Y$, with X, Y being lists.

Compare the lists element by element from left to right; the first differing element determines the order.

Slices of tensors are lists, so we can use the $\leq_{\text{lex}}$ operator.

Breaking index symmetries

Example

$[1, 2, 34, 5, 678] \leq_{\text{lex}} [1, 2, 36, 45, 78]$

because $34 < 36$, even though $678 \not\leq 78$: this $\leq_{\text{lex}}$ order on 1d integer arrays does not require *all* elements to be less than or equal to the corresponding element in the other array; It only considers the first differing pair.

Definition (LEXLESS() and LEXLESSEQ() global constraints)

The LEXLESS(X, Y) (resp. LEXLESSEQ(X, Y)) constraint, where X and Y are same-length lists (or 1D arrays) of decision variables, with size n , holds if and only if X is lexicographically less (resp. less or equal) to Y :

- ▶ either $n = 0$,
- ▶ or $X_1 < Y_1$,
- ▶ or $X_1 = Y_1$ & LEXLESS($[X_i \forall i \in [2..n]]$, $[Y_j \forall j \in [2..n]]$).
 - ▶ resp. $X_1 = Y_1$ & LEXLESSEQ($[X_i \forall i \in [2..n]]$, $[Y_j \forall j \in [2..n]]$).

Breaking index symmetries

Lexicographic ordering constraints along **one** dimension of an array break the index symmetry of that dimension.

Example (Latin Squares)

A latin square of size $n * n$ requires that each row and column contain each number from 1 to n exactly once.

A solution

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

Swap rows 1 and 2

2	3	4	1
1	2	3	4
3	4	1	2
4	1	2	3

Swap columns 1 and 2

2	1	3	4
3	2	4	1
4	3	1	2
1	4	2	3

Latin squares problem has both row and column symmetries!

Assume variables X_{ij} with $i, j \in [1..n]$. Breaking the row symmetries:

$$\text{LEXLESS}([X_{ij} \forall j \in [1..n]], [X_{i+1,j} \forall j \in [1..n]]) \forall i \in [1..n-1]$$

Lexicographic ordering constraints along **every** dimension with index symmetry of an array have two properties:

- + No symmetry class is lost. At least one solution remains from each class.
- In general, **not** all symmetry classes are left with 1 solution, except if the values of the array are distinct, etc.

Counterexample

Assume full row symmetry and full column symmetry:

the arrays

0	0	1
1	1	0

 and

0	1	1
1	0	0

 have lexicographically ordered rows **and** columns, but are symmetric: each can be transformed into the other by swapping their two rows as well as swapping their first and last columns.

General scheme for breaking all variable symmetries: The **Lex-Leader** scheme (see next slide) generates lexicographic ordering constraints that are not necessarily along the dimensions of an array of decision variables.

It guarantees that all the compositions of all variable symmetries are broken.

The Lex-Leader Scheme

Main idea of the Lex-Leader scheme:

- ▶ For each equivalence class of solutions under our symmetry group, predefine *one* to be the canonical solution.
- ▶ Achieves this by adding constraints that are satisfied by canonical solutions and not by symmetric ones.

For each permutation in a given symmetry group G , Lex-Leader produces one lexicographic constraint.

The set of constraints defined by the Lex-Leader method, using the lexicographic relation $\leq_{\text{lex}}$ in a symmetry group G with a chosen variable ordering V , is

$$\forall \sigma \in G, V \leq_{\text{lex}} \sigma(V)$$

where V is the vector of the variables of the CSP (in a defined order), and $\leq_{\text{lex}}$ is the lexicographic ordering relation

The Lex-Leader Scheme

For **any** group G of **variable** symmetries on the indices of the decision variables $x_1, \dots, x_n$ of domain D , which are not necessarily arranged into a 1d array:

1. Choose an ordering of the decision variables, say $x_1, \dots, x_n$.
2. Choose a lexicographic-ordering relation, say $\text{LEXLESSEQ}()$.
3. For every symmetry $\sigma \in G$, add a constraint enforcing the lexicographic relation

$$\text{LEXLESSEQ}([x_1, \dots, x_n], [\sigma(x_1), \dots, \sigma(x_n)])$$

to the problem model.

4. Simplify the resulting constraints, locally and globally.

This yields **exactly** one solution per symmetry class.

Example (2 * 3 array with full row and column symmetry)

Consider the array

x1	x2	x3
x4	x5	x6

 with full row and column symmetry: Need 2! (from row symmetry) $\cdot 3!$ (from column symmetry) $- 1 = 11$ constraints for the ordering $x_1, x_2, x_3, x_4, x_5, x_6$:

1. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_2, x_1, x_3, x_5, x_4, x_6]$) (for cols 1-2)
2. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_1, x_3, x_2, x_4, x_6, x_5]$) (for cols 2-3)
3. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_4, x_5, x_6, x_1, x_2, x_3]$) (for rows)
4. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_6, x_4, x_5, x_3, x_1, x_2]$) (for rows + cols 2-3, 1-2)
5. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_5, x_6, x_4, x_2, x_3, x_1]$) (for rows + cols 1-2, 2-3)
6. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_4, x_6, x_5, x_1, x_3, x_2]$) (for rows + cols 2-3)
7. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_5, x_4, x_6, x_2, x_1, x_3]$) (for rows + cols 1-2)
8. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_6, x_5, x_4, x_3, x_2, x_1]$) (for rows + cols 1-3)
9. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_3, x_2, x_1, x_6, x_5, x_4]$) (for cols 1-3)
10. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_2, x_3, x_1, x_5, x_6, x_4]$) (for cols 1-2, 2-3)
11. LEXLESSEQ($[x_1, x_2, x_3, x_4, x_5, x_6], [x_3, x_1, x_2, x_6, x_4, x_5]$) (for cols 2-3, 1-2)

Symmetry Breaking

The Lex-Leader Scheme is general, but takes exponential space if there are exponentially many symmetries!

Problem-specific symmetry-breaking constraints may be more beneficial.

Note: Breaking **all** the symmetries may increase the solving time:

- ▶ More propagation time than reduction in search time
- ▶ Break only **some** symmetries, but which ones?

Often problem and model specific.

Outline

1. Introduction
2. Symmetries
3. Symmetry Breaking
 - Symmetry Breaking by Reformulation
 - Symmetry Breaking by Constraints
- 4. Dominance Breaking**
5. Recap

Dominance Breaking

- ▶ Symmetry breaking can be beneficial also for optimization problems over objective function f , since for two symmetric solutions s_1, s_2 , we have $f(s_1) = f(s_2)$
- ▶ But we can *generalize* the concept of symmetry breaking in optimization problems
- ▶ Consider *minimization* problems with objective function f . Once we have solution s_1 , we are no longer interested in solution s_2 if $f(s_1) < f(s_2)$.
- ▶ Many constraint problems exhibit **dominance relations**

Definition (Dominance)

Given a *minimisation* problem with objective function f , a (partial) assignment s_1 **dominates** a (partial) assignment s_2 (written $s_1 \prec s_2$) if: whenever s_2 can be extended to a feasible solution of cost c , there exists a feasible extension of s_1 with cost at most c . In particular, discarding dominated assignments never loses an optimal solution.

Dominance Breaking

Main idea: A (partial) assignment that satisfies the constraints can be forbidden if it is dominated.

- ▶ for any solution that this assignment would lead to, there must be another solution that is equally good or better

Identify dominance relations in the problem that can be used to cut the search space: no need to consider dominated assignments when searching for an optimal solution.

Add **dominance-breaking** constraints to reduce the search space and speed up the solving process.

Example (Dominance)

Consider a simple problem with:

- ▶ Domain: $x_i \in \{1, \dots, 10\}$,
- ▶ Constraint: $\text{ALLDIFFERENT}([x_1, \dots, x_{10}])$,
- ▶ Objective function: $\sum_{i=1}^{10} i \cdot x_i$ to be minimized.

The partial assignment $s_1 = [x_1 = 2, x_2 = 1]$ **dominates** the partial assignment $s_2 = [x_1 = 1, x_2 = 2]$, because:

No matter how we label the remaining variables, the corresponding complete assignments s'_1, s'_2 with same values for $x_3, \dots, x_{10}$, will always have $f(s'_1) < f(s'_2)$.

Example (Dominance Breaking)

Identify the **dominance** relation in the previous example:

We have the objective function $f = \sum_{i=1}^{10} i \cdot x_i$.

To minimize the **objective**, we want higher values for variables with smaller indices so that they have less impact on the total sum.

The following **dominance** exists for this objective, without contradicting the constraint

$$x_i \geq x_j, \forall i < j$$

Due to the `ALLDIFFERENT()` constraint, we can simplify it to

$$x_i > x_j, \forall i < j$$

By enforcing such a **lexicographic order** in our variables, dominated assignments will not be explored during the search process

Outline

1. Introduction
2. Symmetries
3. Symmetry Breaking
 - Symmetry Breaking by Reformulation
 - Symmetry Breaking by Constraints
4. Dominance Breaking
5. Recap

Recap

Often symmetries and dominances exist in the problem, **wasting search effort**

- ▶ Variable and value symmetries.

Different types based on where symmetries are introduced:

- ▶ **problem symmetries** are detectable in *every* instance and model of the problem.
- ▶ **instance symmetries** are detectable in *specific instances* of a problem.
- ▶ **model symmetries** are *not* detectable in every model, but only in specific viewpoints.

Symmetry breaking is a very efficient method to **speed up** the solving process

Powerful lexicographic **global constraints** exist to break symmetries

The **Lex-Leader** scheme offers a general method to break symmetries

Dominance breaking further reduces the search for optimisation problems: forbid dominated (partial) assignments that cannot lead to a better optimum